Лекция 5. Стандарты форматирования сообщений.

1. Почему формат сообщения имеет значение

Когда две системы обмениваются данными, они должны договориться о том, как именно эти данные будут представлены в передаваемом сообщении. Транспортный протокол (TCP, UDP, QUIC) обеспечивает доставку байтов от отправителя к получателю, но он не определяет, что означают эти байты. **Задача форматирования сообщений — описать структуру данных, правила их кодирования и декодирования, а также семантику полей.**

Представьте, что два человека говорят по телефону. Транспортный протокол — это телефонная сеть, которая доставляет звук. Но чтобы понять друг друга, им нужен общий язык, общий словарь, общие правила грамматики. Формат сообщения — это и есть такой язык для программных систем.

Неправильный выбор формата приводит к серьёзным последствиям. Сообщения становятся избыточно большими, что увеличивает нагрузку на сеть и затраты на передачу. Парсинг занимает слишком много времени, что увеличивает задержку. Структура данных оказывается негибкой, и при изменении требований приходится переписывать всё. Совместимость между разными языками программирования теряется, и интеграция превращается в кошмар.

По данным исследований, выбор формата сериализации напрямую влияет на производительность распределённых систем. Бинарные форматы (Protobuf, CBOR, MessagePack) демонстрируют значительные преимущества перед текстовыми (JSON, XML) по скорости кодирования и компактности передаваемых данных. В одном из бенчмарков JSON показал время маршалинга 523 мкс и размер 113 КБ, тогда как Protobuf — 130 мкс и 87 КБ, а XML — 1355 мкс и 132 КБ. Разница в разы, и в высоконагруженных системах она становится критичной.

Формат сообщения (Message Format) — это спецификация, определяющая структуру данных в сообщении, правила их кодирования и декодирования, а также семантику отдельных элементов.

Сериализация (Serialization) — процесс преобразования структуры данных (объекта, записи, структуры) в последовательность байтов для передачи по сети или сохранения на диске.

Десериализация (Deserialization) — обратный процесс: преобразование последовательности байтов обратно в структуру данных.

Схема (Schema) — формальное описание структуры данных: перечень полей, их типы, обязательность, ограничения. Схема может быть явной (описана отдельно) или неявной (подразумевается структурой данных).

Схемная эволюция (Schema Evolution) — способность формата поддерживать изменения схемы без нарушения совместимости: добавление полей, удаление полей, изменение типов.

Совместимость (Compatibility) — способность старой и новой версий формата обмениваться данными. Различают обратную совместимость (новый код читает старые данные) и прямую совместимость (старый код читает новые данные).

Человекочитаемость (Human Readability) — способность формата быть прочитанным и понятым человеком без специальных инструментов.

Детерминизм (Determinism) — способность формата генерировать идентичные байтовые последовательности для одинаковых данных. Это критично для распределённых систем, где важна воспроизводимость.

2. Классификация форматов сообщений

2.1. Текстовые форматы

Текстовые форматы представляют данные в виде символов, которые может прочитать человек. Они удобны для отладки, документирования и ручного редактирования. Однако они проигрывают бинарным форматам в компактности и скорости обработки.

Основные представители: JSON, XML, YAML, TOML.

2.2. Бинарные форматы

Бинарные форматы представляют данные в виде последовательности байтов, оптимизированной для машинной обработки. Они компактнее, быстрее парсятся, но нечитаемы для человека без специальных инструментов.

Основные представители: Protocol Buffers, Apache Avro, Apache Thrift, MessagePack, CBOR, FlatBuffers, Cap'n Proto.

2.3. Схемные и бессхемные форматы

Бинарные форматы делятся на схемные (требуют явного описания схемы) и бессхемные (структура данных кодируется непосредственно в сообщении).

Схемные форматы: Protocol Buffers, Apache Avro, Apache Thrift. Они требуют наличия схемы, что обеспечивает строгую типизацию и компактность, но усложняет начало работы.

Бессхемные форматы: JSON, MessagePack, CBOR. Они не требуют отдельной схемы, что упрощает использование, но может приводить к избыточности и отсутствию контроля типов.

3. **Текстовые форматы**

3.1. JSON (JavaScript Object Notation)

JSON — это лёгкий текстовый формат обмена данными, основанный на синтаксисе JavaScript. Он был представлен Дугласом Крокфордом и Чипом Морнингстаром в 2001 году. JSON организует данные в виде пар ключ-значение и массивов.

Синтаксис JSON:

- Объекты начинаются и заканчиваются символами { и } .

- Массивы начинаются и заканчиваются символами [и].

- Все строки заключаются в двойные кавычки.

- Символы “ внутри строк экранируются как \”.

Преимущества JSON:

- Простота и лёгкость изучения.

- Широкая поддержка во всех языках программирования.

- Нативная поддержка в JavaScript.

- Человекочитаемость.

- Меньший размер по сравнению с XML.

Недостатки JSON:

- Отсутствие строгой типизации: числа, строки, булевы значения различаются не всегда очевидно.

- Нет поддержки комментариев.

- Избыточность: ключи повторяются в каждом объекте.

- Медленный парсинг по сравнению с бинарными форматами.

- Отсутствие схемной эволюции.

Области применения: REST API, веб-приложения, конфигурационные файлы, обмен данными между сервисами.

3.2. XML (eXtensible Markup Language)

XML — это язык разметки, разработанный рабочей группой XML в 1996 году и выпущенный в 1998 году. XML использует теги для разделения атрибутов и данных, образуя древовидную структуру.

Синтаксис XML:

- Каждый тег должен иметь закрывающий тег: <contacts> закрывается </contacts>.

- Одиночные теги закрываются как <person />.

Преимущества XML:

- Широкая распространённость в корпоративных системах и legacy-приложениях.

- Поддержка сложных структур и атрибутов.

- Наличие схем (XSD) для валидации.

- Поддержка пространств имён.

Недостатки XML:

- Избыточность: теги повторяются, размер сообщения значительно больше.

- Сложность парсинга: требуется XML-парсер.

- Медленная обработка: в бенчмарках XML показал время маршалинга 1355 мкс против 523 мкс у JSON.

- Whitespace-чувствительность и сложность синтаксиса.

Области применения: SOAP-веб-сервисы, корпоративные интеграции, legacy-системы, документооборот.

3.3. YAML (YAML Ain't Markup Language)

YAML — это текстовый формат сериализации данных, ориентированный на человекочитаемость. Он часто используется для конфигурационных файлов (Docker Compose, Kubernetes, CI/CD).

Преимущества YAML:

- Очень высокая человекочитаемость.

- Поддержка комментариев.

- Компактность по сравнению с XML.

Недостатки YAML:

- Медленный парсинг: в бенчмарках время маршалинга 5318 мкс .

- Whitespace-чувствительность: ошибки в отступах приводят к неверной интерпретации.

- Ограниченная поддержка в некоторых языках.

4. **Бинарные форматы**

4.1. Protocol Buffers (Protobuf)

Protocol Buffers — это бинарный формат сериализации, разработанный Google. Он требует явного описания схемы в файлах .proto. Protobuf обеспечивает высокую производительность и компактность, особенно в системах, где важны пропускная способность и скорость.

Преимущества Protobuf:

- Высокая скорость сериализации и десериализации .

- Компактный размер сообщений.

- Строгая типизация через схему.

- Поддержка схемной эволюции: можно добавлять поля без нарушения совместимости.

- Генерация кода для множества языков.

- Поддержка потоковой передачи.

Недостатки Protobuf:

- Требуется описание схемы.

- Нечитаем для человека.

- Сложнее в отладке.

4.2. Apache Avro

Apache Avro — это бинарный формат сериализации, разработанный в рамках проекта Apache Hadoop. Используются схемы, описанные в JSON, и поддерживает схемную эволюцию.

Преимущества Avro:

- Компактный бинарный формат.

- Отличная поддержка схемной эволюции.

- Широкая поддержка в экосистеме Big Data (Hadoop, Kafka, Spark).

- Схема может передаваться вместе с данными.

Недостатки Avro:

- Требуется схема.

- Сложнее в использовании, чем JSON.

- Меньшая распространённость вне Big Data.

4.3. MessagePack

MessagePack — это компактный бинарный формат сериализации, представленный в 2009 году. Он похож на JSON по модели данных, но кодирует данные в бинарном виде.

Преимущества MessagePack:

- Компактнее JSON.

- Быстрее JSON.

- Прост в использовании (не требует схемы).

- Широкая поддержка в языках программирования.

Недостатки MessagePack:

- Нечитаем для человека.

- Отсутствие схемной эволюции.

- Ограниченная расширяемость.

4.4. CBOR (Concise Binary Object Representation)

CBOR — это бинарный формат, стандартизованный IETF в RFC 8949. Он разработан для IoT и сред с ограниченными ресурсами.

Преимущества CBOR:

- Очень компактный.

- Детерминированное кодирование (100% байтовая идентичность) .

- Поддержка потоковой обработки.

- Стандартизован IETF.

Недостатки CBOR:

- Меньшая распространённость.

- Нечитаем для человека.

4.5. Apache Thrift

Apache Thrift — это фреймворк для RPC и сериализации, разработанный Facebook. Он включает собственный язык описания интерфейсов (IDL) и поддерживает множество языков.

Преимущества Thrift:

- Высокая производительность.

- Поддержка RPC и сериализации.

- Генерация кода для многих языков.

- Поддержка схемной эволюции.

Недостатки Thrift:

- Сложность настройки.

- Меньшая распространённость, чем Protobuf.

5. Критерии выбора формата сообщений

Выбор формата определяется требованиями системы. Рассмотрим основные критерии.

Производительность. Если система высоконагруженная и задержка критична, выбирайте бинарные форматы: Protobuf, Avro, CBOR. Для низких нагрузок достаточно JSON.

Компактность. Если пропускная способность ограничена (мобильные сети, IoT), выбирайте CBOR, Protobuf или MessagePack. JSON и XML избыточны.

Человекочитаемость. Если важна отладка и ручное редактирование, выбирайте JSON или YAML. Для конфигураций — YAML, для API — JSON.

Схемная эволюция. Если система развивается и требуется добавлять поля без нарушения совместимости, выбирайте Avro или Protobuf. JSON и MessagePack не поддерживают эволюцию.

Совместимость. Если система интегрируется с legacy-приложениями, XML остаётся стандартом. Для веб-API — JSON. Для микросервисов — Protobuf или gRPC.

Экосистема. Если используется Kafka, Avro — естественный выбор. Если gRPC — Protobuf. Если Hadoop — Avro или Parquet.

Требования к детерминизму. Если важна воспроизводимость байтовых последовательностей (блокчейн, распределённые консенсусы), выбирайте CBOR или Protobuf .

6. Стандарты и спецификации

6.1. **Стандарты ISO и ГОСТ**

В промышленных системах форматирование сообщений регулируется стандартами. **Например, ГОСТ 34731—202 определяет форматы обмена данными в торговле, включая форматы 08 (структурированные данные по правилам CII), 09 (двоичные данные), 12 (текстовые идентификаторы элементов).**

Эти стандарты описывают структуру конвертов сообщений, заголовки, разделители полей и элементов данных, а также специальные управляющие знаки (RS, GS, FS, US, EOT).

6.2. **Стандарты IETF**

IETF стандартизует форматы для интернета: JSON (RFC 8259), CBOR (RFC 8949), MessagePack (не стандартизован, но широко используется). CBOR определён в RFC 8949 и поддерживается Intel, Microsoft, Amazon и другими компаниями.

6.3. **Отраслевые стандарты**

В финансовой сфере используются FIX (Financial Information eXchange), в здравоохранении — HL7, в авиации — ARINC. Каждый из этих стандартов определяет собственный формат сообщений.

7. Практические аспекты

7.1. **Типичные ошибки при выборе формата**

Использование JSON для высоконагруженных систем. JSON медленный и избыточный. В системах с миллионами запросов в секунду это критично.

Использование XML для новых проектов. XML устарел, сложен и избыточен. Для новых систем он не рекомендуется, если только нет требований legacy-совместимости.

Отсутствие схемы в Protobuf. Protobuf требует схемы. Если схема не поддерживается в актуальном состоянии, система ломается.

Игнорирование схемной эволюции. Если формат не поддерживает эволюцию, любое изменение структуры требует миграции всех данных.

Выбор формата по хайпу. Не стоит выбирать Protobuf только потому, что он модный, если система простая и JSON вполне достаточно.

7.2. Лучшие практики

Для REST API — JSON. JSON универсален, поддерживается всеми, легко читается.

Для gRPC — Protobuf. Protobuf оптимизирован для gRPC, обеспечивает высокую производительность.

Для Kafka — Avro. Avro интегрирован с Schema Registry, поддерживает эволюцию.

Для IoT — CBOR или MessagePack. Компактные бинарные форматы подходят для ограниченных устройств.

Для конфигураций — YAML. YAML читаем и удобен для ручного редактирования.

Для legacy — XML. Если система уже использует XML, не стоит ломать совместимость.

7.3. Schema Registry

Schema Registry — это сервис для хранения и управления схемами данных. Он используется с Kafka, Avro, Protobuf для обеспечения совместимости схем при эволюции.

Schema Registry позволяет:

- хранить версии схем;

- проверять совместимость при регистрации новой версии;

- предоставлять схему потребителям и производителям;

- обеспечивать эволюцию без нарушения работы системы.

8. Домашнее задание

1. Выберите любое веб-приложение или сервис, которым вы пользуетесь. Определите, какие форматы сообщений оно использует (можно предположить на основе публичной документации API).

2. Обоснуйте, почему для этого приложения выбран именно такой формат. Какие альтернативы могли бы быть использованы и почему они не подходят?

3. Приведите пример данных (например, информация о заказе или профиль пользователя) и представьте их в трёх форматах: JSON, XML, Protocol Buffers (в виде схемы). Сравните размер и читаемость.

4. Опишите, какие преимущества дал бы переход с JSON на Protobuf для выбранного приложения. Какие ограничения могли бы возникнуть?

5. Приведите пример системы, для которой XML остаётся оптимальным выбором. Обоснуйте.